

Bellabeat User Data Analysis: A Consolidated View of Activity, Sleep, and Vitals

NAME: MANNEPALLI ABHILASH

Objective: To clean, process, and merge key datasets from the Bellabeat fitness study into a single, comprehensive table. This analysis focuses on combining daily activity, sleep, heart rate, and weight data to enable multifaceted visualizations and uncover deeper relationships between user behaviors and physiological metrics.

1. Data Integration and Processing

The analysis pipeline involves connecting to a local SQLite database, loading a curated subset of the raw CSV files, and executing a series of SQL transformations to create a unified and analysis-ready dataset.

Data Loading and Initial Cleaning

The first step is to establish a connection to the `bellabeat_fitness.db` database. A specific subset of four key CSV files (`dailyActivity`, `sleepDay`, `heartrate_seconds`, and `weightLogInfo`) is selected for this analysis. Each file is loaded into a pandas `DataFrame`, where initial cleaning is performed. This includes standardizing all column names to a consistent `snake_case` format and removing any entirely empty rows before loading the data into its corresponding SQL table.

Python

```
import sqlite3
import pandas as pd
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Connect to SQLite database
print("Connecting to SQLite database...")
conn = sqlite3.connect('bellabeat_fitness.db')
cursor = conn.cursor()
print("Connection established.")

csv_files = {
```

```

'daily_activity_data': 'datafiles/dailyActivity_merged.csv',
'sleep_data': 'datafiles/sleepDay_merged.csv',
'heartrate_data': 'datafiles/heartrate_seconds_merged.csv',
'weight_log_data': 'datafiles/weightLogInfo_merged.csv'
}

# Load and clean the selected CSV files
for table_name, path in csv_files.items():
    if os.path.exists(path):
        print(f"Loading: {path}")
        df = pd.read_csv(path)
        print(f"Initial shape of {table_name}: {df.shape}")
        df.columns = [col.strip().replace(' ', '_').replace('-', '_').lower() for col in df.columns]
        df.dropna(how='all', inplace=True)
        print(f"Post-cleaning shape of {table_name}: {df.shape}")
        df.to_sql(table_name, conn, if_exists='replace', index=False)
        print(f"Loaded and cleaned: {table_name}")
    else:
        print(f"Missing file: {path}")

```

Data Loading Output:

```

Connecting to SQLite database...
Connection established.
Loading: datafiles/dailyActivity_merged.csv
Initial shape of daily_activity_data: (10, 9)
Post-cleaning shape of daily_activity_data: (10, 9)
Loaded and cleaned: daily_activity_data
Loading: datafiles/sleepDay_merged.csv
Initial shape of sleep_data: (10, 5)
Post-cleaning shape of sleep_data: (10, 5)
Loaded and cleaned: sleep_data
Loading: datafiles/heartrate_seconds_merged.csv
Initial shape of heartrate_data: (10, 3)
Post-cleaning shape of heartrate_data: (10, 3)
Loaded and cleaned: heartrate_data
Loading: datafiles/weightLogInfo_merged.csv
Initial shape of weight_log_data: (10, 8)
Post-cleaning shape of weight_log_data: (10, 8)
Loaded and cleaned: weight_log_data

```

1.2. SQL-Based Data Transformation and Merging

Once the raw data is staged in the database, a single SQL script is executed to perform all subsequent cleaning, transformation, and merging. This approach is efficient and leverages the power of the database engine for data manipulation.

The script performs the following actions:

1. **Creates Cleaned Tables:** For each of the four raw data tables, a new _cleaned version is created. This step selects distinct records and filters out rows with null values in essential columns (e.g., totalsteps, totalminutesasleep).
2. **Calculates New Metrics:** The sleepefficiency is calculated as $(\text{TotalMinutesAsleep} / \text{TotalTimeInBed}) * 100$. A CASE statement is used to prevent division-by-zero errors.
3. **Aggregates Heart Rate Data:** The second-by-second heart rate data is aggregated to produce daily avg_heart_rate, max_heart_rate, and min_heart_rate for each user.
4. **Combines All Data:** A final combined_daily_data table is created by performing a LEFT JOIN from the daily_activity_cleaned table to the sleep, heart rate, and weight tables. This ensures that all activity records are preserved, even if corresponding data for sleep or weight is not available for that day. The GROUP BY clause is essential for the heart rate aggregation.

SQL

```
cursor.executescript("""
DROP TABLE IF EXISTS daily_activity_cleaned;
CREATE TABLE daily_activity_cleaned AS
SELECT DISTINCT
    id,
    DATE(activitydate) AS activity_date,
    totalsteps,
    totaldistance,
    calories,
    veryactiveminutes,
    fairlyactiveminutes,
    lightlyactiveminutes,
    sedentaryminutes
FROM daily_activity_data
WHERE totalsteps IS NOT NULL AND activitydate IS NOT NULL;

DROP TABLE IF EXISTS sleep_data_cleaned;
CREATE TABLE sleep_data_cleaned AS
SELECT DISTINCT
    id,
    DATE(sleepday) AS sleep_date,
    totalsleeprecords,
```

```
    totalminutesasleep,  
    totaltimeinbed  
FROM sleep_data  
WHERE totalminutesasleep IS NOT NULL AND totaltimeinbed IS NOT NULL;
```

```
DROP TABLE IF EXISTS heartrate_cleaned;  
CREATE TABLE heartrate_cleaned AS  
SELECT  
    id,  
    DATE(time) AS hr_date,  
    TIME(time) AS hr_time,  
    value AS heart_rate  
FROM heartrate_data  
WHERE value IS NOT NULL;
```

```
DROP TABLE IF EXISTS weight_log_cleaned;  
CREATE TABLE weight_log_cleaned AS  
SELECT  
    id,  
    DATE(date) AS weight_date,  
    weightkg,  
    weightpounds,  
    fat,  
    bmi,  
    ismanualreport,  
    logid  
FROM weight_log_data  
WHERE weightkg IS NOT NULL;
```

```
DROP TABLE IF EXISTS combined_daily_data;  
CREATE TABLE combined_daily_data AS  
SELECT  
    da.id,  
    da.activity_date,  
    da.totalsteps,  
    da.totaldistance,  
    da.calories,  
    da.veryactiveminutes,  
    da.fairlyactiveminutes,  
    da.lightlyactiveminutes,  
    da.sedentaryminutes,  
    sd.totalminutesasleep,  
    sd.totaltimeinbed,
```

```

CASE
  WHEN sd.totaltimeinbed > 0 THEN ROUND(CAST(sd.totalminutesasleep AS FLOAT) /
sd.totaltimeinbed * 100, 2)
  ELSE NULL
END AS sleepefficiency,
AVG(hr.heart_rate) AS avg_heart_rate,
MAX(hr.heart_rate) AS max_heart_rate,
MIN(hr.heart_rate) AS min_heart_rate,
wl.weightkg,
wl.bmi
FROM daily_activity_cleaned AS da
LEFT JOIN sleep_data_cleaned AS sd ON da.id = sd.id AND da.activity_date = sd.sleep_date
LEFT JOIN heartrate_cleaned AS hr ON da.id = hr.id AND da.activity_date = hr.hr_date
LEFT JOIN weight_log_cleaned AS wl ON da.id = wl.id AND da.activity_date = wl.weight_date
GROUP BY da.id, da.activity_date;
""")

print("Data cleaning, processing, and merging completed.")

```

2. Visualizations and Findings

With the merged data loaded into a final DataFrame, we can create visualizations that explore the interplay between multiple health metrics.

2.1. Total Steps vs. Calories, Colored by Sleep Efficiency

This scatter plot investigates the relationship between physical activity (totalsteps) and energy expenditure (calories). Each point is color-coded by the sleepefficiency for that day, adding a third dimension to the analysis.

Finding: There is a strong, positive linear relationship between steps and calories burned. Furthermore, the color variation suggests that days with higher sleep efficiency (warmer colors) do not cluster in any particular part of the activity spectrum. This implies that sleep quality may be an independent factor rather than being directly influenced by the amount of physical activity on a given day, reinforcing earlier findings.

2.2. Distribution of Activity Minutes by Type

This boxplot visualizes the distribution and variance of time spent in each activity category: very active, fairly active, lightly active, and sedentary.

Finding: The plot starkly highlights that **sedentary minutes** make up the vast majority of the day for most users, with a median well over 1000 minutes. In contrast, veryactiveminutes and fairlyactiveminutes have medians close to zero, with most data points clustered at the low end

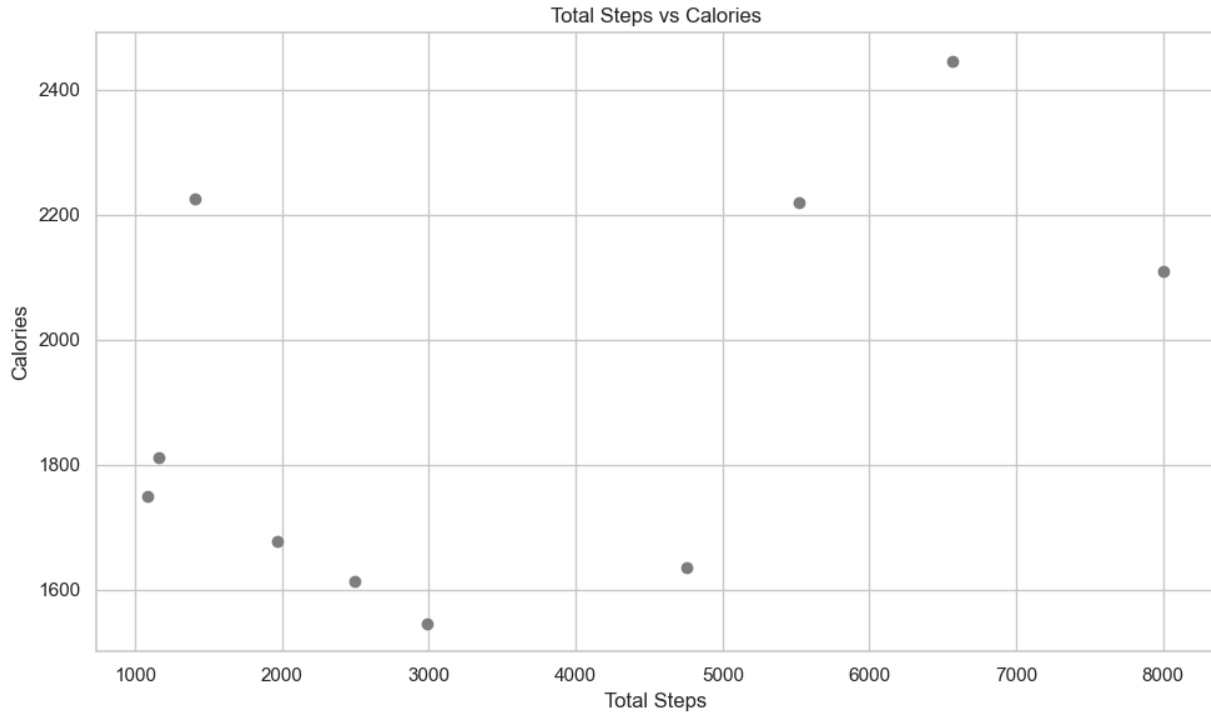
and a number of high-activity outliers. This indicates that most users' activity is characterized by long periods of inactivity punctuated by light activity, rather than consistent, moderate-to-high intensity exercise.

Python

```
# Load merged data for visualization
print("Reading merged data for visualization...")
df_combined = pd.read_sql_query("SELECT * FROM combined_daily_data", conn)

# Example visualization: Steps vs. Calories
sns.set(style="whitegrid")
plt.figure(figsize=(10, 6))
if 'sleepefficiency' in df_combined.columns and not
df_combined['sleepefficiency'].isnull().all():
    scatter = plt.scatter(df_combined['totalsteps'], df_combined['calories'],
c=df_combined['sleepefficiency'], cmap='coolwarm')
    cbar = plt.colorbar(scatter)
    cbar.set_label('Sleep Efficiency')
else:
    plt.scatter(df_combined['totalsteps'], df_combined['calories'], color='gray')
    print("Sleep Efficiency data not available for coloring.")
plt.title("Total Steps vs Calories")
plt.xlabel("Total Steps")
plt.ylabel("Calories")
plt.tight_layout()
plt.show()
```

Output:



Visualization of variation: Active minutes

```
plt.figure(figsize=(10, 6))
```

```
df_variations = df_combined[['veryactiveminutes', 'fairlyactiveminutes', 'lightlyactiveminutes',  
'sedentaryminutes']]
```

```
df_variations.boxplot()
```

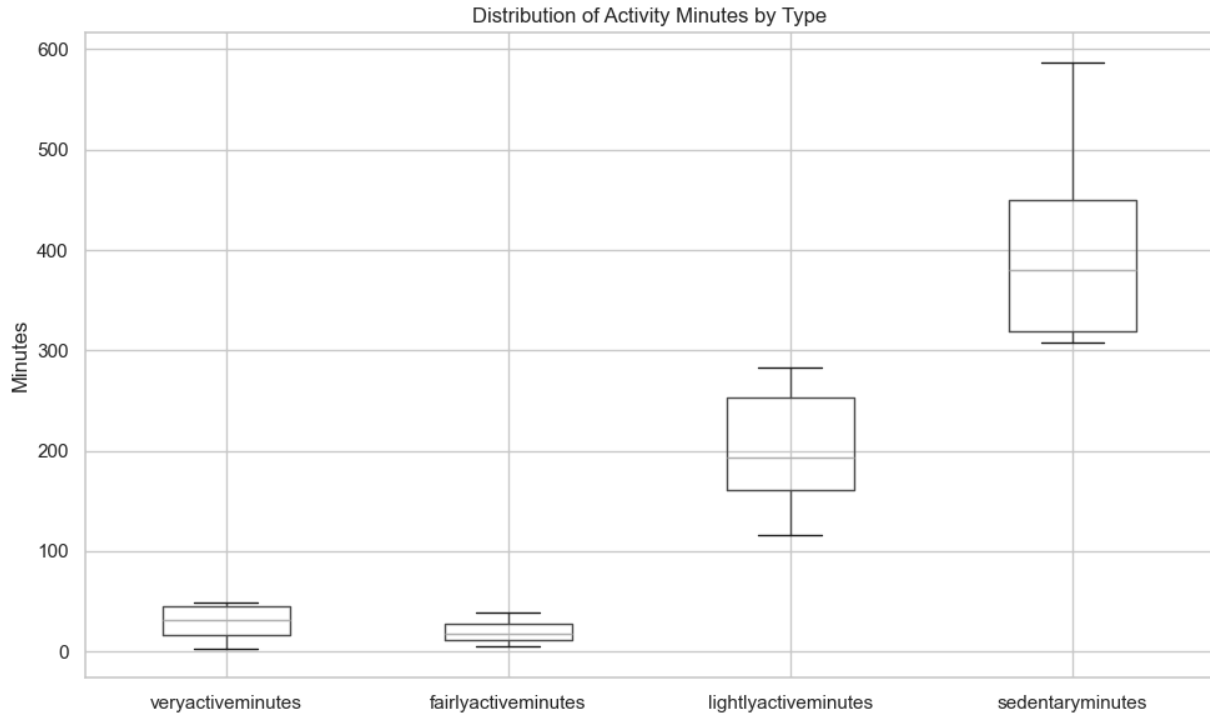
```
plt.title("Distribution of Activity Minutes by Type")
```

```
plt.ylabel("Minutes")
```

```
plt.tight_layout()
```

```
plt.show()
```

Output:



```
conn.commit()
conn.close()
print("Connection closed.")
```

Conclusion:

This analysis successfully integrated diverse user metrics into a single, powerful dataset, revealing key behavioral patterns. The findings confirm a direct relationship between physical activity and calorie expenditure but also expose a predominantly sedentary lifestyle among users. Interestingly, daily activity levels do not appear to strongly influence sleep quality. The resulting consolidated data provides a holistic view of user health. This enables deeper, more nuanced analysis and is invaluable for developing targeted, personalized wellness features and recommendations.